



SBT Framework Guide

Release:



© 2022 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notice>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name:

Product Release:

Document Name: SBT Framework Guide

Document Revision: 25-January-2023

Contents

Support.....	5
1 Overview of the SBT frameworks.....	7
2 Dependency framework.....	11
3 Availability framework.....	13
Availability framework - settings and options.....	14
4 Value dependency framework.....	19
Value dependency framework - settings and options.....	20
5 Existence framework.....	23
Existence framework - settings and options.....	24
6 Schedule framework.....	29
Schedule framework - settings and options.....	32
7 Generic UI framework.....	37
Generic UI framework - settings and options.....	39

Support

For assistance, visit the Guidewire Community.

Guidewire customers

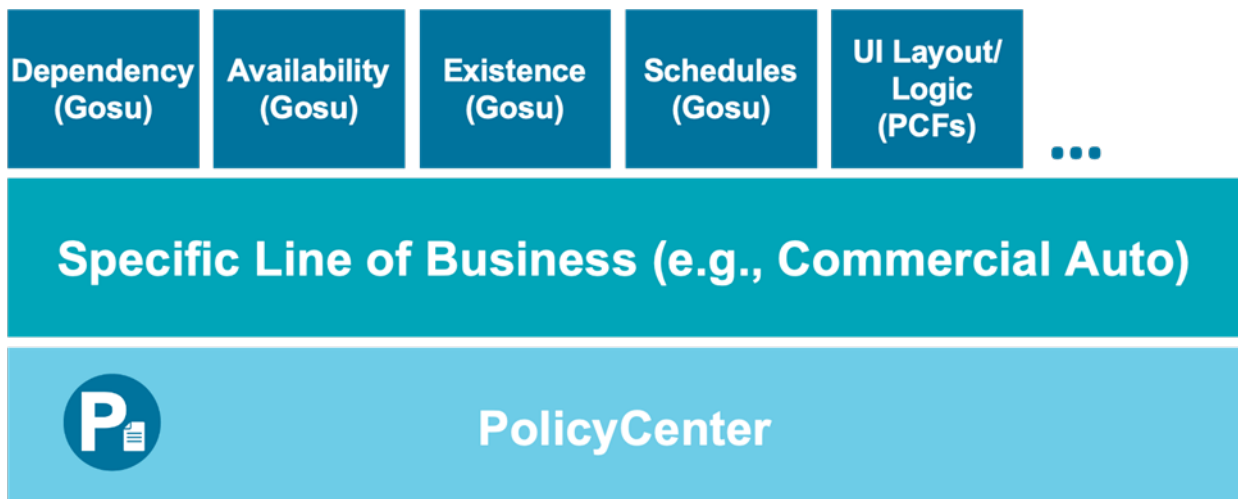
<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Overview of the SBT frameworks

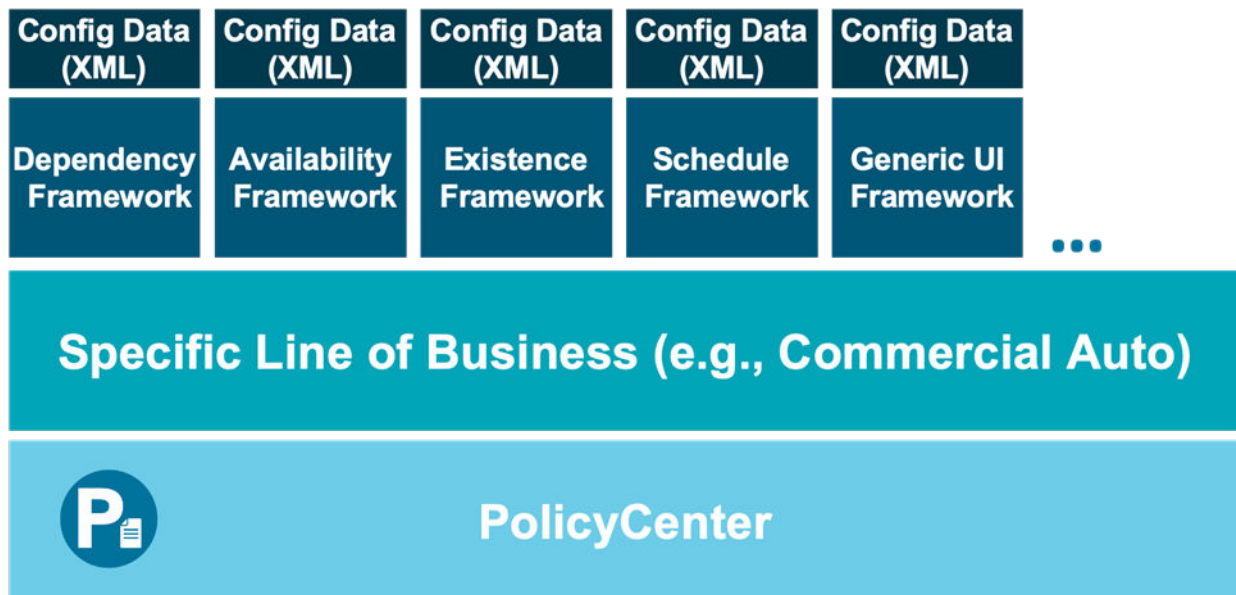
In a typical installation of PolicyCenter, the configuration for a line of business is represented in a number of configuration files, as well as in code. To control the behavior of the line of business, the configuration might include the following components.



Since standards-based templates require frequent updates with ISO electronic rating content, a considerable amount of merging would be required if this configuration was delivered in Gosu code. If in addition your organization had made its own updates, then a three-way merge would be required. This would add to the complexity of the process.

To make it easier for your organization to apply frequent updates to your configuration, Guidewire designed multiple frameworks that externalize the data into XML files. No Gosu code is contained in the updates. This data is easier to merge when new updates arrive, and it also reduces the amount of maintenance that is required.

The Guidewire framework configuration is shown below. The frameworks replace the Gosu components.



The remainder of this guide focuses on the general approach used across the frameworks, as well as specific details about each of the key frameworks. This information will help you to use the frameworks in your implementation.

General approach

For most frameworks, the approach involves two main components:

1. An XML file that describes the desired behavior for the specific framework.
2. The framework code that understands the XML file structure and data.

At most your organization will only need to modify the first component (the XML configuration data) but not the second component (the framework code). Changing the framework code may produce unexpected behavior and might make any future updates from Guidewire unstable. You might not need to change the XML file if the current behavior - the default behavior specified by the ISO ERC - is acceptable to your organization.

Key frameworks

Dependency framework

Specifies items that are dependent on other items (for example one coverage that is dependent on another coverage). Dependencies can be defined for clauses, coverage terms, coverable attributes and scheduled item properties. The dependency framework comprises three smaller independent frameworks:

Availability framework

Defines the availability of clauses (coverages, for example), coverage terms, coverable (risk) attributes, and scheduled item properties.

Existence framework

Defines whether an item is mandatory or optional (referred to as **existence** within PolicyCenter). Existence can be defined for clauses, coverage terms, coverable attributes, and scheduled item properties.

Value dependency framework

This framework determines the availability of coverage term options.

Schedule framework

This framework is used to define the structure of a schedule in metadata instead of code.

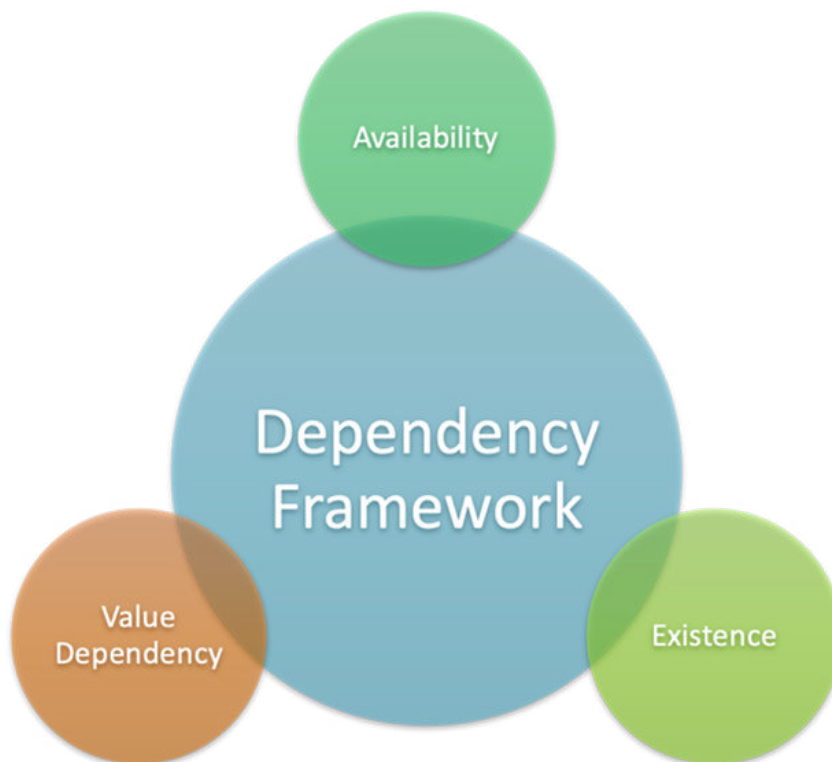
Generic UI framework

Specifies which attributes to display on the user interface, primarily for coverable (risk) attributes. Since ISO defines risk attributes differently by state, this framework is used to avoid any state-specific Gosu code within a PCF.

Dependency framework

The dependency framework specifies certain items within PolicyCenter that are dependent on other items. For example, choosing one coverage often makes another coverage also required, or alternatively prevents another coverage from being chosen together with the first coverage. These dependencies in PolicyCenter are often represented through Gosu code, and this is similar to the ability within Product Designer to define dependencies.

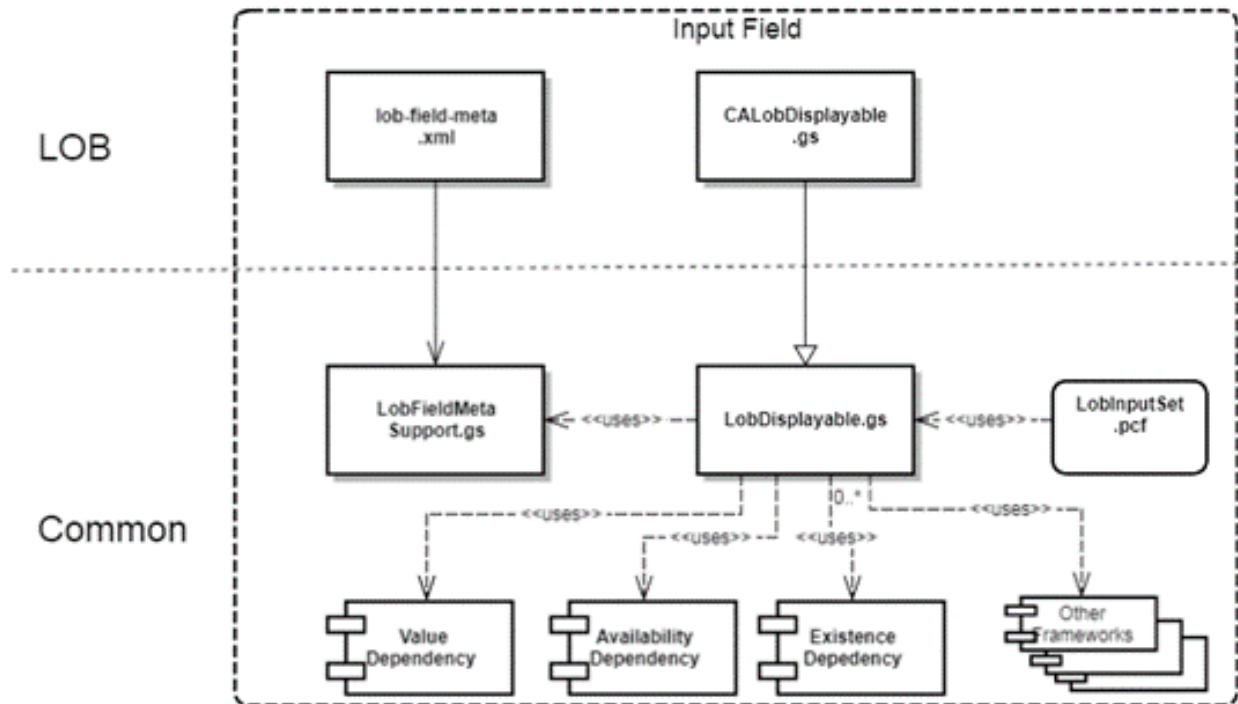
The dependency framework consists of three individual frameworks, as shown in the following diagram:



The dependency framework unites these individual frameworks through the user interface of each line of business. The user interface determines, for each field or value, whether it is "displayable" by calling each of the frameworks as needed and determining if the field or value is to appear. The decision is based on each of the frameworks and might

include factors such as the state (jurisdiction) the current coverable is in, the other coverables chosen, or the effective date of the policy.

From a technical viewpoint, the interaction with the various frameworks is shown in the following UML diagram. This diagram is included only for reference purposes.



Each of these frameworks is discussed in the following sections.

Availability framework

The availability framework defines the availability of clauses (coverages, for example), coverage terms, coverable (risk) attributes and scheduled item properties. This feature is similar to the ability within Product Designer to define availability. For standards-based templates, Guidewire has created availability framework Gosu code that is called by each item. The Gosu code then reads the file `xxx-availability-config.xml`, where "xxx" is the product name for the line. For example, the file name for Commercial Auto is `ca7-availability-config.xml`.

XML file structure for availability

This file can be found at `Config/<line>-availability-config.xml`. An extract of the file is shown below.

```
<AvailabilityConfig>
  <Container name="CA7SpecialTypesRepossessedAutoLiabPremiumToReachMi" type="Clause">
    <Version>
      <EffectiveDate>2018-11-01</EffectiveDate>
      <Depends>
        <On>
          classPath="gw.lob.ca7.custom.availability.cw.v11012018.CA7SpecialTypesRepossessedAutoLiabPremiumToReachMinAv1">
            <classVariable>CA7CommAutoLine/CA7SpecialType/Clazz</classVariable>
          </On>
        </Depends>
      </Version>
    </Container>
    <Container name="CA7FireTheftAndLmtdSpecifiedCausesOfLossCovForDGRD" type="Clause">
      <Version>
        <Jurisdiction>NY</Jurisdiction>
        <EffectiveDate>2019-07-02</EffectiveDate>
        <Depends>
          <On>
            classPath="gw.lob.ca7.availability.generated.cw.v11022018.CA7FireTheftAndLmtdSpecifiedCausesOfLossCovForDGRDAv1">
              <classVariable>CA7CommAutoLine/CA7LinePolicyType</classVariable>
              <classVariable>CA7CommAutoLine/CA7AutoDealer/CA7VehicleOthThanCollisionGRD/CA7CovType14</classVariable>
            </On>
          </Depends>
        </Version>
      </Container>
    </AvailabilityConfig>
```

XML schema definition for availability

The XSD for `xxx-availability-config.xml` is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="AvailabilityConfig" type="AvailabilityConfigType"/>
  <xs:complexType name="AvailabilityConfigType">
    <xs:complexContent>
      <xs:extension base="AbstractConfigType"/>
    </xs:complexContent>
  </xs:complexType>
  <xs:complexType name="AbstractConfigType" abstract="true">
    <xs:sequence>
      <xs:element name="Container" type="ContainerType" minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

```

</xs:sequence>
</xs:complexType>
<xs:complexType name="ContainerType">
  <xs:sequence>
    <xs:element name="Version" type="VersionType" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="Property" type="PropertyType" minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="name" type="xs:string" use="required"/>
  <xs:attribute name="type" type="ContainerTypeEnum" use="required"/>
</xs:complexType>
<xs:complexType name="PropertyType">
  <xs:sequence>
    <xs:element name="Version" type="VersionType" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="name" type="xs:string" use="required"/>
</xs:complexType>
<xs:complexType name="VersionType">
  <xs:sequence>
    <xs:element name="Jurisdiction" type="xs:string" minOccurs="0" maxOccurs="1"/>
    <xs:element name="EffectiveDate" type="xs:date" minOccurs="1" maxOccurs="1"/>
    <xs:element name="ExpiryDate" type="xs:date" minOccurs="0" maxOccurs="1"/>
    <xs:element name="Depends" type="DependsType" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element name="Available" type="xs:boolean" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="DependsType">
  <xs:sequence>
    <xs:element name="On" type="OnType" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="OnType">
  <xs:sequence>
    <xs:element name="classVariable" type="xs:string" minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="function" type="xs:string" use="optional"/>
  <xs:attribute name="path" type="xs:string"/>
  <xs:attribute name="op" type="xs:string" use="optional"/>
  <xs:attribute name="value" type="xs:string" use="optional"/>
  <xs:attribute name="valuePath" type="xs:string" use="optional"/>
  <xs:attribute name="classPath" type="xs:string" use="optional"/>
</xs:complexType>
<xs:simpleType name="ContainerTypeEnum">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Coverable"/>
    <xs:enumeration value="Schedule"/>
    <xs:enumeration value="Clause"/>
  </xs:restriction>
</xs:simpleType>
</xs:schema>

```

Availability framework - settings and options

This section explains the meaning of each component within the `availability-config.xml` file.

Availability Configuration

The `<AvailabilityConfig>` and `</AvailabilityConfig>` are the outermost tags. Everything within these tags defines the availability for this line of business. These tags must not be removed or edited, and they must not be added anywhere else in the XML file.

Container

The `Container` tag defines the item for which availability is being assessed.

```
<Container name="CA7SpecialTypesRepossessedAutoLiabPremiumToReachMi" type="Clause">
```

In the example below, everything within the `Container` block defines the availability of the **Business Interruption** clause (coverage).

The valid values for the `type` attribute are:

- Clause
- Coverage
- Schedule

Each of these values refers to the corresponding item within PolicyCenter.

Property

When there is a Property block inside a Container block, it changes the identity of the element that is being configured. Instead of referring to an entire clause, coverable, or schedule, the configuration within a Property tag refers to a specific attribute within that clause, coverable or schedule. The following code shows an example of a Property block:

```
<Container name="CA7BusinessInterruption" type="Clause">
  <Property name="CA7ComprehensiveCovWanted">
    <Version>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Depends>
        <On path="/CA7CommAutoLine/CA7BusinessInterruption/CA7SpecifiedCausesOfLossCovWanted" op="=" value="No"/>
      </Depends>
    </Version>
  </Property>
```

In this example, the attribute CA7ComprehensiveCovWanted is available if CA7SpecifiedCausesOfLossCovWanted is equal to No.

Version

Many attributes and aspects of a particular item (for example, a coverage) change over time as ISO changes the ERC. To manage this situation, there is a <Version> tag. Within a Version tag, there can be several effective date segments defined by their Start Date. As an example, you might see ("..." implies that one or more XML lines unnecessary to this example have been suppressed):

```
<Property name="CA7GaragingLocation1">
  <Version>
    <EffectiveDate>2018-11-02</EffectiveDate>
    ...
  </Version>
  <Version>
    <EffectiveDate>2019-07-02</EffectiveDate>
    ...
  </Version>
</Property>
```

In this case, the configuration in the first Version block will be honored starting on November 2nd, 2018 through to July 1st, 2019, while the second Version tag configuration will be used from July 2nd, 2019 onwards (the last entry, by default, has no End Date).

See the next definition for details on the Effective Date tag.

Effective Date

The EffectiveDate tag can only be used within a Version block, and everything inside a specific Version block is the configuration that will be used from the date defined within the EffectiveDate start/end tags (specified in yyyy-mm-dd format) until the date defined in the next Version block. If there is no subsequent Version block, the end date is determined to be blank (this means that the last Version block configuration will be used from the effective date forward).

Depends

The Depends block defines the things that the Primary Item (as defined in the Container block) is dependent upon for its availability. Within each Depends block are one or more On blocks that define the dependent items. As an example, you might see the following ("..." implies that one or more XML lines unnecessary to this example have been suppressed):

```
<Depends>
  <On .../>
  <On .../>
</Depends>
```

In this example, the Primary Item (as defined in the Container block) needs to meet the criteria of both of the On blocks within (for example, multiple On tags are treated as a logical AND).

At other times, you might see the following:

```
<Depends>
  <On .../>
</Depends>
<Depends>
  <On .../>
</Depends>
```

In this example, the **Primary Item** (as defined in the **Container** block) needs to meet the criteria of both of the **Depends** tags within, each of which have a single **On** tag within them. In this example, the two **Depends** tags are treated as a logical OR. The details of the **On** tag are covered in the next definition.

On

The **On** block defines a logical statement that is assessed to determine the availability of the **Primary Item**. A sample is shown below.

```
<Depends>
  <On path="/CA7CommAutoLine/CA7GrossReceiptsOrMileageBasisMedPay/CA7MedPay1" op="=" value="Yes"/>
  <On path="/CA7CommAutoLine/CA7LineGrossReceiptsRatingBasis" op="=" value="Yes"/>
</Depends>
```

In this example, the availability depends on two things (which are assessed as an AND as outlined in the previous section). Firstly, if the value for **Gross Receipts or Mileage Basis Medical Payments** is equal to **Yes** and the value for **Gross Receipts Rating Basis** is equal to **Yes**, then this **Depends** block is assessed to **True**. If this is the only **Depends** block for this **Primary Item**, then the **Primary Item** is deemed as available.

Within the **On** block, there are four settings:

path

This setting defines a specific attribute within the current coverable or coverage based on its path in the **PolicyCenter** policy graph (the name of which is based on the path in the ISO ERC).

op

This is the logical operator that is being used (valid options are "=", "!", ">" and "<").

value

This is the comparison value used to assess the logical statement, expressed as a **String**.

function

Although not shown in the above example, the **function** attribute provides functions beyond the logical comparisons of the **op** attribute. An example of a function is **exist**, which evaluates whether the item defined by the **path** attribute exists on the current coverable. This function can be used as shown in the following example:

```
<Depends>
  <On path="/CA7CommAutoLine/CA7PrivatePassenger/CA7PrivatePassengerAddedPersonalInjuryProtectionPPT"
    function="exist"/>
</Depends>
```

Calling Gosu code from an On block

Another less common variation of an **On** statement is one that calls Gosu code and then assesses the results of calling that code. This type of **On** statement is shown in the example below:

```
<Depends>
  <On classPath="gw.lob.ca7.custom.availability.cw.v11012018.CA7SpecialTypesReposessedAutoLiabPremiumToReachMinAv1">
    <classVariable>/CA7CommAutoLine/CA7SpecialType/Clazz</classVariable>
  </On>
</Depends>
```

In this case, **On** becomes a block and includes a tag within called **classVariable**. In this case, the availability framework calls the Gosu class at the defined path and tests the result of the variable **/CA7CommAutoLine/CA7SpecialType/Clazz**. If this value is **True** and this is the only **Depends** block, then the **Primary Item** will be available.

Comparing two attributes in an On tag

The final (and rare within ISO lines) variation of the On tag compares two values within the policy graph. This variation is shown in the following example:

```
<Depends>
  <On path="/CA7CommAutoLine/CA7DrivingSchool/CA7NumberOfDrivingInstructors" op=">" valuePath="/CommercialAutomobile/CommercialAutomobileDrivingSchoolsNonOwnedCoverages/NumberOfOwnedAutosUsedForDriverTraining"/>
</Depends>
```

In this example, instead of comparing the item specified by path to a constant such as Yes, the comparison is to another item on the policy. In this case, if the Number of Driving Instructors is greater than the Number of Owned Autos Used for Driver Training, then the Primary Item from the Container block will be available.

Jurisdiction

The Jurisdiction tag, which is always found within a Version block, specifies that the configuration within the Version block only applies to that specific jurisdiction. The following example shows a typical Version block with a Jurisdiction within it:

```
<Version>
  <Jurisdiction>NY</Jurisdiction>
  <EffectiveDate>2019-07-02</EffectiveDate>
  <Depends>
    <On .../>
  </Depends>
</Version>
```

In this case, the availability for the Primary Item is assessed against this configuration only if the Primary Item is on a New York-based coverable. The only valid content within the **Jurisdiction** tag is a standard two-letter US state abbreviation.

Value dependency framework

The value dependency framework specifies certain items within PolicyCenter that are dependent on other items. For example, choosing one coverage often makes another coverage also required, or alternatively prevents another coverage from being chosen together with the first. These dependencies in PolicyCenter are often represented through Gosu code. This is equivalent to the ability within Product Designer to define dependencies through X,Y, Z.

For standards-based templates, Guidewire has created value dependency framework Gosu code that is called by each item that, in turn, reads the file xxx-value-dependency-config.xml. In the file name "XXX" is the product name for the line, so for Commercial Auto the file name is ca7-value-dependency-config.xml.

XML file structure for value dependency

This file can be found at Config/<line>-value-dependency-config.xml.

A small section of the file is shown below:

```
<ValueDependency property="RatingBase" container="CA7AutoDealer">
  <SystemTable name="CA7GaragekeepersRatingBas">
    <EffectiveDate>2018-11-02</EffectiveDate>
    <KeyColumn name="GaragekeepersCollisionCovType">
      <Path>/CA7CommAutoLine/CA7AutoDealer/
        CA7GaragekeepersCovCollisionGRD/CA7CovType</Path>
    </KeyColumn>
    <KeyColumn name="Garagekeepers0thThanCollisionC">
      <Path>/CA7CommAutoLine/CA7AutoDealer/
        CA7GaragekeepersCov0thThanCollisionGRD/CA7CovType4</Path>
    </KeyColumn>
  </SystemTable>
</ValueDependency>
```

XML schema definition (XSD) for value dependency

The XSD for xxx-value-dependency-config.xml is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="ValueDependencyConfig" type="ValueDependencyConfigType"/>
  <xs:element name="Path" type="xs:string"/>
  <xs:element name="ClassPath" type="xs:string"/>
  <xs:element name="ValueColumn" type="xs:string"/>
  <xs:element name="KeyColumn" type="KeyColumn"/>
  <xs:element name="SystemTable" type="SystemTable"/>
  <xs:element name="ValueDependency" type="ValueDependency"/>
  <xs:element name="Jurisdiction" type="xs:string"/>
  <xs:element name="EffectiveDate" type="xs:date"/>
  <xs:element name="ExpiryDate" type="xs:date"/>
  <xs:element name="Available" type="xs:boolean"/>
  <xs:attribute name="name" type="xs:string"/>
  <xs:attribute name="property" type="xs:string"/>
  <xs:attribute name="container" type="xs:string"/>
  <xs:complexType name="ValueDependencyConfigType">
```

```

<xs:sequence>
  <xs:element ref="ValueDependency" maxOccurs="unbounded"/>
</xs:sequence>
</xs:complexType>
<xs:complexType name="ValueDependency">
  <xs:sequence>
    <xs:element ref="SystemTable" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute ref="property" use="required"/>
  <xs:attribute ref="container" use="required"/>
</xs:complexType>
<xs:complexType name="SystemTable">
  <xs:sequence>
    <xs:element ref="EffectiveDate"/>
    <xs:element ref="ExpiryDate" minOccurs="0"/>
    <xs:element ref="Jurisdiction" minOccurs="0"/>
    <xs:element ref="KeyColumn" minOccurs="0" maxOccurs="unbounded"/>
    <xs:element ref="ValueColumn" minOccurs="0" maxOccurs="1"/>
    <xs:element ref="Available" minOccurs="0"/>
  </xs:sequence>
  <xs:attribute ref="name" use="required"/>
</xs:complexType>
<xs:complexType name="KeyColumn">
  <xs:choice>
    <xs:element ref="Path"/>
    <xs:element ref="ClassPath"/>
  </xs:choice>
  <xs:attribute ref="name" use="required"/>
</xs:complexType>
</xs:schema>

```

Value dependency framework - settings and options

Value Dependency Configuration

The `<ValueDependencyConfig>` and `</ValueDependencyConfig>` tags are outermost. Everything within these tags defines value dependencies for this line of business. These tags must not be removed or edited, nor must they be added anywhere else in the XML file

Value Dependency

The `ValueDependency` block is the main construct within the value dependency framework. Everything within a single `ValueDependency` block represents one specific dependency between items within the standards-based template. The `ValueDependency` entity defines the field with values and identifies the system table containing the values. An example is shown below:

```

<ValueDependency property="TrailerHowUsed" container="CA7Truck">
  <SystemTable name="CA7TTTTTrailerSemitrailerU">
    <EffectiveDate>2018-11-02</EffectiveDate>
    <KeyColumn name="PolicyType">
      <Path>/CA7CommAutoLine/CA7LinePolicyType</Path>
    </KeyColumn>
    <KeyColumn name="SizeClass">
      <Path>/CA7CommAutoLine/CA7Truck/SizeClass</Path>
    </KeyColumn>
  </SystemTable>
</ValueDependency>

```

There are two attributes within the `ValueDependency` opening tag:

Property

The property with the value list or drop-down.

Container

The entity or clause containing the property.

The contents of the `ValueDependency` block are detailed in the following sections.

System Table

The `SystemTable` block defines the behavior of that specific system table. The name of the system table (from the name attribute) refers to the entity name of the table within PolicyCenter. The following XML components can be found within a `SystemTable` block.

Name

The entity name of the system table. Cross-reference this entity name with the system table XML to find out the file name of the system table.

KeyColumn

If the list is dependent on other values, the KeyColumn defines the dependent value. The following attributes/blocks are found within the KeyColumn block:

name

The name of the dependent column.

Path

The policy period path to the dependent value (the mapping to the dependent key column).

Existence framework

The existence framework defines whether an item is mandatory or optional (termed **Existence** within PolicyCenter). Existence can be defined for clauses, coverage terms, coverable attributes and scheduled item properties. This is equivalent to the ability within Product Designer to define existence through X, Y, Z. For standards-based templates, Guidewire has created existence framework Gosu code. This code is called by each item that in turn reads the file xxx-existence-config.xml. In the file name the xxx is the product name for the line, so for example for Commercial Auto the file name will be ca7-existence-config.xml.

XML file structure for existence

The XML file existence can be found here: Config/<line>-existence-config.xml.

The sample of the file is shown below.

```
<Container name="CA7CovForInjuryToLeasedWorkers" type="Clause">
  <Property name="CA7ManualPremium3">
    <Version>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Existence met="Required"/>
    </Version>
  </Property>
</Container>
<Container name="CA7CustomerComplaintLegalDefense" type="Clause">
  <Property name="CA7CustomerComplaintLegalDefenseAggLimit">
    <Version>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Existence met="Electable"/>
    </Version>
  </Property>
  <Property name="CA7CustomerComplaintLegalDefenseLimit">
    <Version>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Existence met="Electable"/>
    </Version>
  </Property>
</Container>
```

XML schema definition (XSD) for existence

The XSD for the xxx-existence-config.xml is shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:redefine schemaLocation="availability-config.xsd">
    <xs:complexType name="VersionType">
      <xs:complexContent>
        <xs:extension base="VersionType">
          <xs:sequence>
            <xs:element name="Existence" type="ExistenceType" minOccurs="0" maxOccurs="1"/>
          </xs:sequence>
        </xs:extension>
      </xs:complexContent>
    </xs:complexType>
  </xs:redefine>
</xs:schema>
```

```

        </xs:complexContent>
    </xs:complexType>
</xs:redefine>
<xs:element name="ExistenceConfig" type="ExistenceConfigType"/>
<xs:complexType name="ExistenceConfigType">
    <xs:complexContent>
        <xs:extension base="AbstractConfigType"/>
    </xs:complexContent>
</xs:complexType>
<xs:complexType name="ExistenceType">
    <xs:attribute name="met" type="ExistenceTypeEnum" use="required"/>
    <xs:attribute name="notMet" type="ExistenceTypeEnum" use="optional"/>
</xs:complexType>
<xs:simpleType name="ExistenceTypeEnum">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Electable"/>
        <xs:enumeration value="Required"/>
        <xs:enumeration value="Suggested"/>
    </xs:restriction>
</xs:simpleType>
</xs:schema>

```

Existence framework - settings and options

This section explains the purpose of each component within the availability-config.xml file.

Existence Configuration

Everything between the <ExistenceConfig> and </ExistenceConfig> tags defines existence for this line of business. These tags must not be removed or edited, and they must not be added anywhere else in the XML file.

Container

The <Container> tag defines the item for which existence is being assessed.

```
<Container name="CA7SpecialTypesRepossessedAutoLiabPremiumToReachMi" type="Clause">
```

In the example below, everything within the Container block defines the existence of the clause (coverage) called Business Interruption:

```
<Container name="CA7BusinessInterruption" type="Clause">
```

The valid values for the type attribute are listed below:

- Clause
- Coverable
- Schedule

Each of these values refers to the corresponding item within PolicyCenter.

Property

The Property block is important within the existence framework because it identifies the item for which the existence is being determined. An example of a Property block is shown below.

```

<Container name="CA7BusinessInterruption" type="Clause">
    <Property name="CA7DurationOfWaitingPeriod">
        <Version>
            <EffectiveDate>2018-11-02</EffectiveDate>
            <Existence met="Required"/>
        </Version>
    </Property>

```

In this example, after November 2nd, 2018, the attribute Duration of Waiting Period (within the Business Interruption coverage) is required in a policy and in the user interface

Version

Many attributes and aspects of a particular item (for example, a coverage) change over time as ISO changes the ERC. The <Version> tag is used to manage this situation. Within a Version block, there can be several effective date segments defined by their Start Date. As an example, you might see the following:

```
<Property name="CA7GaragingLocation">
  <Version>
    <EffectiveDate>2018-11-02</EffectiveDate>
    ...
  </Version>
  <Version>
    <EffectiveDate>2019-07-02</EffectiveDate>
    ...
  </Version>
</Property>
```

In this case, the configuration in the first Version block is honored starting on November 2nd, 2018 through to July 1st, 2019, while the second Version block configuration will be used from July 2nd, 2019 onwards (the last entry, by default, has no End Date).

The EffectiveDate tag is described below.

Effective Date

The EffectiveDate tag can only be used within a Version block. A specific Version block contains the configuration that will be used from the date defined within the EffectiveDate start/end tags (specified in yyyy-mm-dd format) until the date defined in the next Version block. If there is no subsequent Version block, the end date is determined to be blank (the last Version block configuration will be used from the Effective Date forward).

Existence

The Existence tag is the main tag within the existence framework. It defines whether the attribute listed is required or optional (using the PolicyCenter terminology of required or electable). A sample Property block using the Existence tag is shown below.

```
<Property name="CA7LocationsAndOpsMedPayLimit">
  <Version>
    <EffectiveDate>2018-11-02</EffectiveDate>
    <Existence met="Electable"/>
  </Version>
</Property>
<Property name="CA7ManualPremium">
  <Version>
    <EffectiveDate>2018-11-02</EffectiveDate>
    <Existence met="Required"/>
  </Version>
</Property>
```

In this example, the attribute CA7LocationsAndOpsMedPayLimit is optional on a policy (and the user interface), while the attribute CA7ManualPremium is required.

Existence with Depends

A more complex version of Existence tag usage occurs when you add a Depends block. This is similar to using the Depends block for availability. An example is shown below:

```
<Container name="CA7ExperienceRatingModification" type="Clause">
  <Property name="CA7ExpenseVariationApplies">
    <Version>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Depends>
        <On path="/CA7CommAutoLine/
          CA7LineExperienceRatingModificationApplies"
          op="=" value="Yes"/>
      </Depends>
      <Existence met="Required" notMet="Electable"/>
    </Version>
  </Property>
```

In this example, the existence of the Property called CA7ExpenseVariationApplies is dependent on the evaluation of a logical statement – specifically, whether Experience Rating Modification is set to Yes or No. If it is Yes, then

CA7ExpenseVariationApplies is required, otherwise it is optional. In this case, the new attribute notMet is introduced.

More about Depends

The Depends block defines the things that the Primary Item (the item defined in the Container block) is dependent upon for its existence. Within each Depends block will be one or more On blocks that define the dependent items. As an example, you might see the following:

```
<Depends>
  <On .../>
  <On .../>
</Depends>
```

In this example, the Primary Item (as defined in the Container block) needs to meet the criteria of both of the On blocks within it (for example, multiple On tags are treated as a logical AND).

At other times, you might see the following:

```
<Depends>
  <On .../>
</Depends>
<Depends>
  <On .../>
</Depends>
```

In this example, the Primary Item (as defined in the Container block) needs to meet the criteria of both of the Depends blocks within it, each of which has a single On tag. In this example, the two Depends blocks are treated as a logical OR.

On

The On block defines a logical statement that is executed to determine the existence of the Primary Item. A sample On block is shown below.

```
<Depends>
  <On path="/CA7CommAutoLine/CA7GrossReceiptsOrMileageBasisMedPay/CA7MedPay1" op="=" value="Yes"/>
  <On path="/CA7CommAutoLine/CA7LineGrossReceiptsRatingBasis" op="=" value="Yes"/>
</Depends>
```

In this example, the Existence depends on two things (which are assessed as an AND, as outlined in the previous section). Firstly, if the value for Gross Receipts or Mileage Basis Medical Payments is equal to Yes and the value for Gross Receipts Rating Basis is equal to Yes, then this Depends block will be assessed to True. If this is the only Depends block for this Primary Item, then the Primary Item will be deemed as Available.

Within the On block, there are four settings:

path

This setting defines a specific attribute within the current coverable or coverage based on its path in the PolicyCenter policy graph (the name of which is based on the path in the ISO ERC).

op

This is the logical operator used (valid options are "=", "!=", ">" and "<").

value

This is the comparison value used to assess the logical statement, expressed as a String.

function

Although not shown in the above example, the function attribute provides functions beyond the logical comparisons of the op attribute. One such function is exist, which simply evaluates whether the item defined by the path attribute exists on the current coverable.

An example usage of the function attribute is shown below:

```
<Depends>
  <On path="/CA7CommAutoLine/CA7PrivatePassenger/CA7PrivatePassengerAddedPersonalInjuryProtectionPPT"
```

```
function="exist"/>
</Depends>
```

The valid function attributes are listed below:

exist

Tests for the existence of an item.

Calling Gosu code from an On block

A less common variation of an On statement calls Gosu code and then assesses the results of running that code. An example of this type of On statement is shown below:

```
<Depends>
  <On classPath="gw.lob.ca7.custom.existence.cw.v11012018.CA7SpecialTypesRepossessedAutoLiabPremiumToReachMinAv1">
    <classVariable>/CA7CommAutoLine/CA7SpecialType/Clazz</classVariable>
  </On>
</Depends>
```

In this case, On becomes a block and includes a tag within it called `classVariable`. In this case, the Existence Framework will call the Gosu class at the path defined and test the results of the variable `/CA7CommAutoLine/CA7SpecialType/Clazz`. If this value is True and this is the only Depends block, the Primary Item will be available.

Comparing two attributes in an On tag

The final variation of the On tag compares two values within the policy graph. An example of this variation of the On tag is shown below:

```
<Property name="CA7GaragingLocation">
  <Version>
    <Jurisdiction>NY</Jurisdiction>
    <EffectiveDate>2019-07-02</EffectiveDate>
    <Depends>
      <On path="/CA7CommAutoLine/CA7DrivingSchool/CA7NumberOfDrivingInstructors" op=">" valuePath="/CommercialAutomobile/CommercialAutomobileDrivingSchoolsNonOwnedCoverages/NumberOfOwnedAutosUsedForDriverTraining"/>
    </Depends>
    <Existence met="Required" notMet="Electable"/>
  </Version>
</Property>
```

In this example, instead of comparing the item specified by path to a constant such as Yes, the comparison is to another item on the policy. In this case, if the Number of Driving Instructors is greater than the Number of Owned Autos Used for Driver Training, then the Property referenced in the Property block (in this case, Garaging Location), is required. If not, it will be optional (as specified in the met and notMet attributes).

Jurisdiction

The Jurisdiction tag, which is always found within a Version block, specifies that the configuration within the Version block only applies to that specific Jurisdiction. The following example shows a typical Version block containing a Jurisdiction tag.

```
<Property name="CA7GaragingLocationOverride">
  <Version>
    <Jurisdiction>NY</Jurisdiction>
    <EffectiveDate>2019-07-02</EffectiveDate>
    <Depends>
      <On path="/CA7CommAutoLine/CA7DrivingSchool/CA7GaragingLocation" op="=" value="Other"/>
    </Depends>
    <Existence met="Required" notMet="Electable"/>
  </Version>
</Property>
```

In this case, the Existence for the GaragingLocationOverride property will be assessed against this configuration only if it is on a New York-based coverable. The only valid content within the Jurisdiction tag is a standard two letter US state abbreviation.

Schedule framework

The schedule framework allows you to define a schedule in metadata instead of through code.

The process of using the schedule framework is described below:

1. Use the generic scheduled item entity `ScheduledItem.eti` from base `PolicyCenter` to define the generic columns. The generic scheduled item is used to persist the item property values.
2. Use the line-specific `ScheduledItem.eti` (for example `CA7ScheduledItem.eti`) if the base `PolicyCenter ScheduledItem.eti` does not have enough columns.
3. Use `Config/<line>-schedule-config.xml` to define the schedule. Note that all scheduled item properties support state deviations (for example, the default value for an item property can be a state-deviated value).

Schedule items entity file

A sample scheduled items is shown below (`CA7ScheduledItem.eti` from Commercial Auto):

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<delegate requiresType="effdated" extendable="true" name="CA7ScheduledItem" xmlns="http://guidewire.com/datamodel">
  <fulldescription>A CA7 scheduled item</fulldescription>
  <column default="false" name="BoolCol3" nullok="true" type="bit"/>
  <column default="false" name="BoolCol4" nullok="true" type="bit"/>
  <column desc="Decimal field" name="DecimalCol1" nullok="true" type="decimal">
    <columnParam name="scale" value="2"/>
    <columnParam name="precision" value="10"/>
  </column>
  <column name="LongStringCol1" nullok="true" type="longtext"/>
  <column name="LongStringCol2" nullok="true" type="longtext"/>
  <column desc="Non-negative integer field" getterScriptability="doesNotExist" name="NonNegativeIntCol1" nullok="true"
setterScriptability="doesNotExist" type="nonnegativeinteger"/>
  <column desc="Non-negative integer field" getterScriptability="doesNotExist" name="NonNegativeIntCol2" nullok="true"
setterScriptability="doesNotExist" type="nonnegativeinteger"/>
  <column desc="Non-negative integer field" getterScriptability="doesNotExist" name="NonNegativeIntCol3" nullok="true"
setterScriptability="doesNotExist" type="nonnegativeinteger"/>
  <column desc="Non-negative integer field" getterScriptability="doesNotExist" name="NonNegativeIntCol4" nullok="true"
setterScriptability="doesNotExist" type="nonnegativeinteger"/>
  <column desc="Option field" name="OptionCol1" nullok="true" type="shorttext"/>
  <column desc="Option field" name="OptionCol2" nullok="true" type="shorttext"/>
  <column desc="Option field" name="OptionCol3" nullok="true" type="shorttext"/>
  <column desc="Option field" name="OptionCol4" nullok="true" type="shorttext"/>
  <column desc="Option field" name="OptionCol5" nullok="true" type="shorttext"/>
  <column desc="Option field" name="OptionCol6" nullok="true" type="shorttext"/>
  <column desc="String field" name="StringCol3" nullok="true" type="shorttext"/>
  <column desc="String field" name="StringCol4" nullok="true" type="shorttext"/>
  <foreignkey fkentity="PolicyAddlInsuredDetail" name="AdditionalInsured" nullok="true"/>
</delegate>
```

XML file structure for the schedule framework

The file can be found at Config/<line>-schedule-config.xml. A sample of this file is shown below.

```
<Clause pattern="CA7DesignatedInsured">
  <AutoNumberPropertyInfo name="ScheduleNumber">
    <ColumnName>ScheduleNumber</ColumnName>
    <ColumnLabel>Web.Policy.Schedule.ScheduleNumber</ColumnLabel>
    <Required>true</Required>
    <Priority>1</Priority>
  </AutoNumberPropertyInfo>
  <PropertyInfo name="NameOfPersonsOrOrganizations">
    <Version>
      <ColumnName>StringCol1</ColumnName>
      <ColumnLabel>Web.Policy.Schedule.CA7DesignatedInsured.
        NameOfPersonsOrOrganizations</ColumnLabel>
      <Required>false</Required>
      <Priority>100</Priority>
      <Type>String</Type>
      <ScheduledItemType>ScheduledItem</ScheduledItemType>
      <IsKey>true</IsKey>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Available>true</Available>
    </Version>
  </PropertyInfo>
</Clause>
```

XML schema definition (XSD) for the schedule framework

The XSD for xxx-schedule-config.xml is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- simple types -->
  <xs:simpleType name="TypeListType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z_][a-zA-Z\d_]*"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="ScheduledItemTypeType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z_][a-zA-Z\d_]*"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="NameType">
    <xs:restriction base="xs:string">
      <xs:pattern value="([a-zA-Z\d_])*"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="ColumnNameType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z_][a-zA-Z\d_]*"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="PropertyInfoEnum">
    <xs:restriction base="xs:string">
      <xs:enumeration value="Integer"/>
      <xs:enumeration value="Percent"/>
      <xs:enumeration value="String"/>
      <xs:enumeration value="Boolean"/>
      <xs:enumeration value="Date"/>
      <xs:enumeration value="TypeKey"/>
      <xs:enumeration value="ForeignKey"/>
      <xs:enumeration value="PolicyLocation"/>
      <xs:enumeration value="Building"/>
      <xs:enumeration value="NamedInsured"/>
      <xs:enumeration value="ForeignKeyWithOptionLabels"/>
      <xs:enumeration value="AdditionalInsured"/>
      <xs:enumeration value="AdditionalInterest"/>
      <xs:enumeration value="Decimal"/>
      <xs:enumeration value="ReadOnly"/>
      <xs:enumeration value="TextArea"/>
      <xs:enumeration value="Option"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="ItemClauseType">
    <xs:attribute ref="pattern" use="required"/>
    <xs:attribute ref="amendmentOf" use="optional"/>
    <xs:attribute ref="whenClause" use="optional"/>
    <xs:attribute ref="hideFromSummary" use="optional"/>
  </xs:complexType>
  <xs:simpleType name="PatternType">
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-zA-Z_][a-zA-Z\d_]*"/>
    </xs:restriction>
  </xs:simpleType>
```

```

<xs:complexType name="ScheduleConfigType">
  <xs:sequence>
    <xs:element ref="Clause" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<!-- complex types -->
<xs:complexType name="ClauseType">
  <xs:sequence>
    <xs:choice minOccurs="1" maxOccurs="unbounded">
      <xs:element ref="AutoNumberPropertyInfo" minOccurs="1" maxOccurs="1"/>
      <xs:element ref="PropertyInfo" minOccurs="0" maxOccurs="unbounded" />
    </xs:choice>
    <xs:element ref="ItemClause" minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute ref="pattern" use="required"/>
</xs:complexType>
<xs:complexType name="PropertyInfoType">
  <xs:sequence>
    <xs:element ref="Version" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute ref="name" use="required" />
</xs:complexType>
<xs:complexType name="BasePropertyInfoType">
  <xs:sequence>
    <xs:element ref="ColumnName" minOccurs="0" maxOccurs="1"/>
    <xs:element ref="ColumnLabel" minOccurs="0" maxOccurs="1"/>
    <xs:element ref="Required" minOccurs="0" maxOccurs="1"/>
    <xs:element ref="Priority" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="PropertyInfoVersionType">
  <xs:complexContent>
    <xs:extension base="BasePropertyInfoType">
      <xs:sequence>
        <xs:element ref="Type" minOccurs="1" maxOccurs="1"/>
        <xs:element ref="ScheduledItemType" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="Default" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="IsKey" minOccurs="0" maxOccurs="1"/>
        <!-- Decimal -->
        <xs:element ref="Minimum" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="Maximum" minOccurs="0" maxOccurs="1"/>
        <!-- ReadOnly -->
        <xs:element ref="MethodExpression" minOccurs="0" maxOccurs="1"/>
        <!-- accessible variables:
        'entity' - current ScheduledItem
        'prop' - current IEntityPropertyInfo build based on 'ScheduledItemType' & 'ColumnName'-->
        <!-- TypeKey -->
        <xs:element ref="TypeList" minOccurs="0" maxOccurs="1"/> <!-- minOccurs="1" TODO try xsd 1.1
conditional feature -->
        <xs:element ref="TypeFilter" minOccurs="0" maxOccurs="1"/>
        <!-- ForeignKey and ForeignKeyWithOptionLabels -->
conditional feature <xs:element ref="ValueRangeGetter" minOccurs="0" maxOccurs="1"/> <!-- minOccurs="1" TODO try xsd 1.1
        <!-- ForeignKeyWithOptionLabels -->
        <xs:element ref="OptionGroupLabelMethodExpression" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="OptionLabelMethodExpression" minOccurs="0" maxOccurs="1"/>
        <!-- version specific -->
        <xs:element ref="Jurisdiction" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="EffectiveDate" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="ExpirationDate" minOccurs="0" maxOccurs="1"/>
        <xs:element ref="Available" minOccurs="0" maxOccurs="1"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:complexType name="AutoNumberPropertyInfoType">
  <xs:complexContent>
    <xs:extension base="BasePropertyInfoType">
      <xs:attribute ref="name" use="required" />
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
<xs:attribute name="name" type="NameType"/>
<xs:attribute name="min" type="xs:decimal"/>
<xs:attribute name="max" type="xs:decimal"/>
<xs:attribute name="pattern" type="PatternType"/>
<xs:attribute name="amendmentOf" type="xs:string"/>
<xs:attribute name="whenClause" type="xs:string"/>
<xs:attribute name="hideFromSummary" type="xs:boolean"/>
<xs:element name="Type" type="PropertyInfoEnum"/>
<xs:element name="Priority" type="xs:int"/>
<xs:element name="IsKey" type="xs:boolean"/>
<xs:element name="Minimum" type="xs:int"/>
<xs:element name="Maximum" type="xs:int"/>
<xs:element name="ValueRangeGetter" type="xs:string"/>
<xs:element name="ScheduledItemType" type="ScheduledItemTypeType"/>
<xs:element name="TypeFilter" type="xs:string"/>
<xs:element name="TypeList" type="TypeListType"/>
<xs:element name="ScheduleConfig" type="ScheduleConfigType"/>
<xs:element name="Required" type="xs:boolean"/>

```

```

<xs:element name="Default" type="xs:string"/>
<xs:element name="MethodExpression" type="xs:string"/>
<xs:element name="PropertyInfo" type="PropertyInfoType"/>
<xs:element name="Version" type="PropertyInfoVersionType"/>
<xs:element name="AutoNumberPropertyInfo" type="AutoNumberPropertyInfoType"/>
<xs:element name="ColumnName" type="ColumnNameType"/>
<xs:element name="ColumnLabel" type="xs:string"/>
<xs:element name="Clause" type="ClauseType"/>
<xs:element name="ItemClause" type="ItemClauseType"/>
<xs:element name="OptionGroupLabelMethodExpression" type="xs:string"/>
<xs:element name="OptionLabelMethodExpression" type="xs:string"/>
<xs:element name="Jurisdiction" type="xs:string"/>
<xs:element name="EffectiveDate" type="xs:string"/>
<xs:element name="ExpirationDate" type="xs:string"/>
<xs:element name="Available" type="xs:boolean"/>
</xs:schema>

```

Schedule framework - settings and options

ScheduleConfig

`<ScheduleConfig>` and `</ScheduleConfig>` are the outermost tags – everything within them defines schedules for this line of business. These tags must not be removed, edited, or added anywhere else in the XML file.

Clause

The Clause block is the most important element within the schedule configuration. It defines the clause (for example, a scheduled coverage) and also how the schedule items appear and are handled within the schedule. A sample of a Clause block is shown below:

```

<Clause pattern="CA7BusinessInterruption">
  <AutoNumberPropertyInfo name="ScheduleNumber">
    <ColumnName>ScheduleNumber</ColumnName>
    <ColumnLabel>Web.Policy.Schedule.ScheduleNumber</ColumnLabel>
    <Required>true</Required>
    <Priority>1</Priority>
  </AutoNumberPropertyInfo>
  <PropertyInfo name="DecriptionOfScheduledPropertyOptionA">
    <Version>
      <ColumnName>LongStringCol1</ColumnName>
      <ColumnLabel>Web.Policy.Schedule.CA7BusinessInterruption.
        DecriptionOfScheduledPropertyOptionA</ColumnLabel>
      <Required>false</Required>
      <Priority>600</Priority>
      <Type>TextArea</Type>
      <ScheduledItemType>CA7ScheduledItem</ScheduledItemType>
      <EffectiveDate>2018-11-02</EffectiveDate>
      <Available>true</Available>
    </Version>
  </PropertyInfo>
</Clause>

```

The Clause tag contains three important tags:

- AutoNumberPropertyInfo
- PropertyInfo
- ItemClause

Each of these tags is described in the following sections.

AutoNumberPropertyInfo

This XML block creates an automatically-generated integer (starting at 1) to be used to number the items within the schedule. A sample of an AutoNumberPropertyInfo block is shown below:

```

<ScheduleConfig>
  <Clause pattern="CA7AdditionalInsuredGeneralLiabCovsCoOwnerOfInsure">
    <AutoNumberPropertyInfo name="ScheduleNumber">
      <ColumnName>ScheduleNumber</ColumnName>
      <ColumnLabel>Web.Policy.Schedule.ScheduleNumber</ColumnLabel>
      <Required>true</Required>
      <Priority>1</Priority>
    </AutoNumberPropertyInfo>
  </Clause>
</ScheduleConfig>

```

In this example, the schedule (in the user interface and in the underlying entity) creates a sequential integer ID for each item on the schedule and stores it in the field ScheduleNumber.

The elements within the `AutoNumberPropertyInfo` block are listed below:

ColumnName

The schedule database column name from the ETI.

ColumnLabel

The display key for the label.

Required

Whether this **AutoNumber** field is required (default set to `True`).

Priority

Item priority controls the order of appearance on the user interface (by default, it is set to 1).

PropertyInfo

The `PropertyInfo` block is used to define the content of each field (other than the `AutoNumber` field) within a schedule. A sample `PropertyInfo` block is shown below.

```
<PropertyInfo name="NameOfPersonOrOrganization">
  <Version>
    <ColumnName>StringCol2</ColumnName>
    <ColumnLabel>Web.Policy.Schedule.
      CA7AdditionalInsuredGeneralLiabCovsCoOwnerOfInsure.
      NameOfPersonOrOrganization</ColumnLabel>
    <Required>false</Required>
    <Priority>100</Priority>
    <Type>TextArea</Type>
    <ScheduledItemType>ScheduledItem</ScheduledItemType>
    <IsKey>true</IsKey>
    <EffectiveDate>2018-11-02</EffectiveDate>
    <Available>true</Available>
  </Version>
</PropertyInfo>
```

The elements within the `PropertyInfo` block are described below:

ColumnName

The database column name from the schedule ETI.

ColumnLabel

The display key for the label.

Required

Whether this field is required.

Priority

The order in which items appear in the UI.

Type

Controls the type of UI widget used for the property (based on PCF Types). This is used within the page `ScheduledItemPropertyInputSet.pcf`, using `schedulePropertyInfo.ValueType` or `type` to determine the control. The valid types are listed below:

Type	ScheduledItemPropertyInputSet control	PCF control
autonumber	IntegerInputSet	Input control
Integer	IntegerInputSet	Input control
Percent	IntegerInputSet	Input control
String	StringInputSet	Input control
Boolean	BooleanInputSet	CheckBoxInput
Date	DateInputSet	Input control
TypeKey	RangeInputSet	RangeInput

Type	ScheduledItemPropertyInputSet control	PCF control
ForeignKey	RangeInputSet	RangeInput
ForeignKeyWithOptionLabels	RangeInputSet	RangeInput
AdditionalInsured	AdditionalInsuredInputSet	Input control
AdditionalInterest	AdditionalInterestInputSet	Input control
Decimal	BigDecimalInputSet	Input control
TextArea	TextAreaInputSet	TextAreaInput
Option	RangeInputSet	RangeInput

ScheduledItemType

The PolicyCenter scheduled item entity (as an example, for CA7, this is CA7ScheduledItem).

IsKey

Indicates if this property is a key for the scheduled item property.

Minimum

Decimal or integer only - the minimum value allowed in the field.

Maximum

Decimal or integer only - the maximum value allowed in the field.

TypeList

The TypeKey for the property.

ValueRangeGetter

ForeignKey and ForeignKeyWithOptionLabels.

OptionGroupLabelMethodExpression

ForeignKeyWithOptionLabels

OptionLabelMethodExpression

ForeignKeyWithOptionLabels

Jurisdiction

The state that the PropertyInfo applies to (using the standard two-character state code, for example NY). If the jurisdiction is absent, then the PropertyInfo applies to all states (countrywide).

EffectiveDate

The date when this entry take effect (specified in yyyy-mm-dd format).

ExpirationDate

The date when this entry expires (specified in yyyy-mm-dd format).

Available

A boolean value that indicates whether the column is available (appears on the screen) during the time period specified by the EffectiveDate and ExpirationDate.

ItemClause

The ItemClause tag specifies the clauses that are available for the scheduled coverage/clause. The following example shows how the ItemClause is used.

```
<Clause pattern="CA7BusinessInterruption">
  <AutoNumberPropertyInfo name="ScheduleNumber">
    ...
  </AutoNumberPropertyInfo>
  <PropertyInfo name="DescriptionOfScheduledPropertyOptionA">
    ...
  </PropertyInfo>
  <PropertyInfo name="DescriptionOfScheduledPropertyOptionB">
    ...
  </PropertyInfo>
</Clause>
```

```
<ItemClause pattern="CA7BusinessInterruptionCollision"/>
<ItemClause pattern="CA7BusinessInterruptionComprehensive"/>
<ItemClause pattern="CA7BusinessInterruptionSpecifiedCausesOfLoss"/>
</Clause>
```

In this case, three coverages are available for the schedule, specified by their clause ID in PolicyCenter. The options with the ItemClause tag are listed below:

- pattern
- amendmentOf
- whenClause
- hideFromSummary

Version

Many attributes and aspects of a particular item (for example, a coverage) change over time as ISO changes the ERC. To manage this, there is a <Version> tag. Within a Version block, there can be several effective date segments defined by their start date. An example is shown below:

```
<PropertyInfo name="NameOfPersonOrOrganization">
  <Version>
    <EffectiveDate>2018-11-02</EffectiveDate>
    ...
  </Version>
  <Version>
    <EffectiveDate>2019-07-02</EffectiveDate>
    ...
  </Version>
</PropertyInfo>
```

EffectiveDate

The EffectiveDate tag can only be used within a Version block. Everything inside a specific Version block is the configuration that will be used from the date defined within the EffectiveDate start/end tags (specified in yyyy-mm-dd format) until the date defined in the next Version block. If there is no subsequent Version block, the end date is determined to be blank (the last Version block configuration will be used from the effective date forward).

Jurisdiction

The Jurisdiction tag, which is always found within a Version block, specifies that the configuration within the Version block only applies to that specific Jurisdiction. The following example shows a typical Version block with a Jurisdiction tag within it.

```
<PropertyInfo name="AccidentPreventionDiscount">
  <Version>
    <ColumnName>...</ColumnName>
    <ColumnLabel>...</ColumnLabel>
    <Required>...</Required>
    <Priority>...</Priority>
    <Type>...</Type>
    <ScheduledItemType>...</ScheduledItemType>
    <Default>...</Default>
    <Jurisdiction>NY</Jurisdiction>
    <EffectiveDate>...</EffectiveDate>
    <Available>...</Available>
  </Version>
```

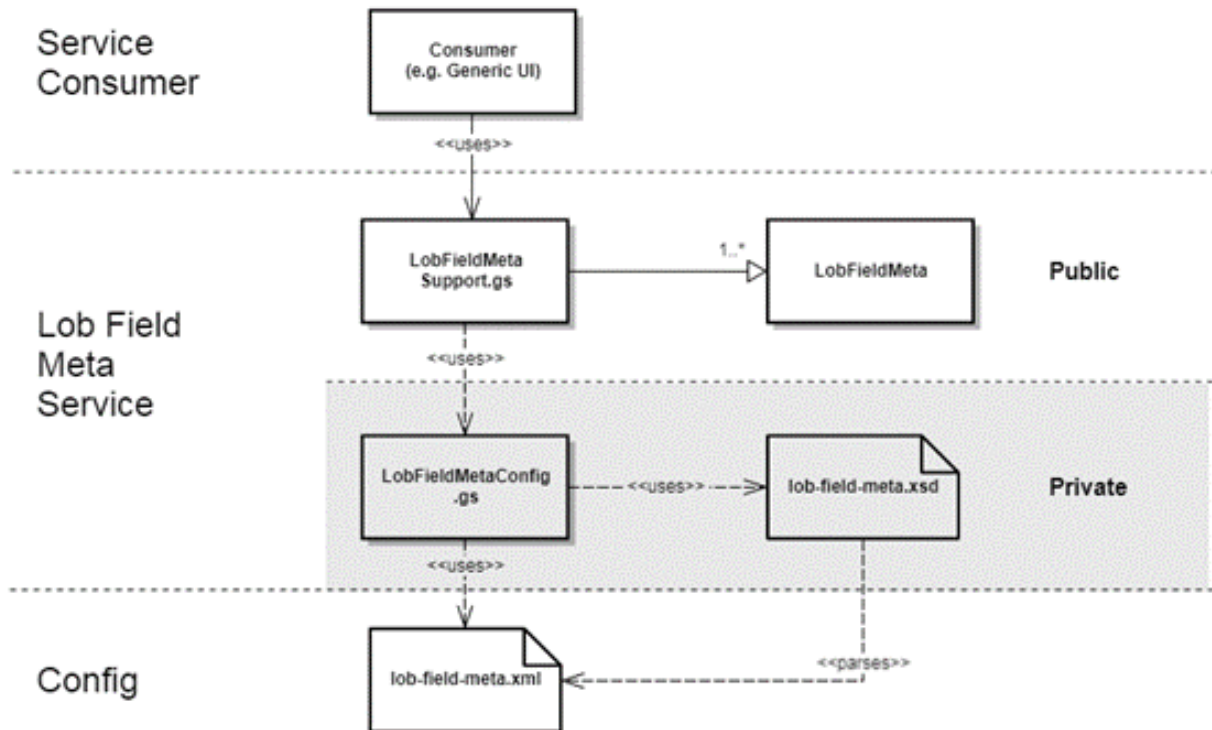

Generic UI framework

The generic UI framework supports the user interface presentation of coverable (risk) fields and allows those fields to be treated differently by state (which is required to support ISO-based lines of business). This replaces the use of PCF modal views for state-level variations.

The following table defines the key components used with configuration-driven widgets. The shaded items must not be modified and are only shown here for reference and troubleshooting purposes.

Category	Repo	Component	Distribution
Configuration	LOB	lob-field-meta.xml	Field metadata definition.
Schema	Common	lob-field-meta.xsd	Schema for lob-field-meta.xml.
Data Structure	Common	LobFieldMeta.gs	Data structure to store field metadata.
Service	Common	LobFieldMetaSupport.gs	Service to query field metadata.
View Model	Common	LobDisplayable.gs	Displayable class that backs the Generic UI framework. This class coordinates with all frameworks and dependencies to provide all UI required properties.
Widget	Common	LobEntityDV.pcf	Widget to show entity properties that are defined in LOB field metadata. Use LobInputSet to show the input.
Widget	Common	LobInputSet.pcf	Widget for the input field.

The diagram below shows how the schema of lob-field-meta.xml configuration file underpins the generic UI Framework. This diagram is provided for reference purposes.



XML file structure for the generic UI framework

The main file that controls the generic UI framework is xxx-lob-field-meta.xml. This file can be found at Config/<line>-lob-field-meta.xml.

A sample of the file is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<LobFieldMeta>
  <Entity name="CA7CommAutoLine">
    <Column name="CA7LineTerrorismCoverage">
      <Version>
        <Label>entity.CA7CommAutoLine.CA7LineTerrorismCoverage</Label>
        <WidgetType>Range</WidgetType>
        <EffectiveDate>2018-11-02</EffectiveDate>
        <Priority>600</Priority>
        <Editable>true</Editable>
        <Available>true</Available>
        <Visible>true</Visible>
        <Required>true</Required>
        <DefaultValue>No</DefaultValue>
      </Version>
    </Column>
    <Column name="CA7LineExperienceRatingModificationApplies">
      <Version>
        <Label>entity.CA7CommAutoLine.CA7LineExperienceRatingModificationApplies</Label>
        <WidgetType>Range</WidgetType>
        <EffectiveDate>2018-11-02</EffectiveDate>
        <Priority>7200</Priority>
        <Editable>true</Editable>
        <Available>true</Available>
        <Visible>true</Visible>
        <Required>false</Required>
        <DefaultValue>No</DefaultValue>
      </Version>
    </Column>
  </Entity>
</LobFieldMeta>
```

XML schema definition (XSD) for the generic UI framework

The XSD for xxx-lob-field-meta.xml is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

```

<xs:element name="LobFieldMeta" type="LobFieldMetaType"/>
<xs:complexType name="LobFieldMetaType">
  <xs:sequence>
    <xs:element name="Entity" type="EntityType" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:complexType name="EntityType">
  <xs:sequence>
    <xs:element name="Column" type="ColumnType" minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="name" type="xs:string" use="required"/>
</xs:complexType>
<xs:complexType name="ColumnType">
  <xs:sequence>
    <xs:element name="Version" type="VersionType" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
  <xs:attribute name="name" type="xs:string" use="required"/>
</xs:complexType>
<xs:complexType name="VersionType">
  <xs:sequence>
    <xs:element name="Jurisdiction" type="xs:string" minOccurs="0"/>
    <xs:element name="Label" type="xs:string" minOccurs="0"/>
    <xs:element name="WidgetType" type="xs:string" minOccurs="0"/>
    <xs:element name="EffectiveDate" type="xs:date" minOccurs="0"/>
    <xs:element name="ExpiryDate" type="xs:date" minOccurs="0"/>
    <xs:element name="Priority" type="xs:integer" minOccurs="0"/>
    <xs:element name="Editable" type="xs:boolean" minOccurs="0"/>
    <xs:element name="Available" type="xs:boolean" minOccurs="0"/>
    <xs:element name="Visible" type="xs:boolean" minOccurs="0"/>
    <xs:element name="Required" type="xs:boolean" minOccurs="0"/>
    <xs:choice minOccurs="0" maxOccurs="1">
      <xs:element name="DefaultValue" type="xs:string"/>
      <xs:element name="DefaultPath" type="xs:string"/>
    </xs:choice>
    <xs:element name="Max" type="xs:string" minOccurs="0"/>
    <xs:element name="Min" type="xs:string" minOccurs="0"/>
    <!-- deprecated -->
    <xs:element name="ForceRefresh" type="xs:boolean" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
</xs:schema>

```

Generic UI framework - settings and options

LobFieldMeta

<LobFieldMeta> and </LobFieldMeta> are the outer tags – everything within these tags defines the generic UI framework behavior for this line of business. These tags must not be removed or edited, and they must not be added anywhere else in the XML file.

Entity

The Entity block is the main block for the generic UI framework. Everything within a specific Entity block is the defined behavior for that specific coverable (which is represented in PolicyCenter as an entity with .eti and .etx file). Within the Entity block there are one or more Column blocks that define each field in the coverable.

```

<Entity name="CA7AutoDealer">
  <Column name="AccidentPreventionDiscount">
    ...
  </Column>
  <Column name="ClassCode">
    ...
  </Column>
  ...
</Entity>

```

Column

The Column block encompasses the configuration for a specific field (column) with the defined coverable. The content can be versioned (see the Version block section below).

A sample of the file is shown below.

```

<Column name="CA7LineTerrorismCoverage">
  <Version>
    <Label>entity.CA7CommAutoLine.CA7LineTerrorismCoverage</Label>
    <WidgetType>Range</WidgetType>
  </Version>
</Column>

```

```

    <EffectiveDate>2018-11-02</EffectiveDate>
    <Priority>600</Priority>
    <Editable>true</Editable>
    <Available>true</Available>
    <Visible>true</Visible>
    <Required>true</Required>
    <DefaultValue>No</DefaultValue>
  </Version>
</Column>

```

The Column block attributes are described below.

Jurisdiction

Availability Data - used to select availability by state, if not present then used by countrywide, the jurisdiction overrides countrywide based on the effective dates.

Label

PCF attribute - display key used for the label.

WidgetType

Controls the type of PCF widget used (WidgetType = LobFieldWidgetType).

The options for WidgetType are shown in the table below.

WidgetType	LobInputSet control	PCF control
BooleanRadio	LobBooleanRadioInputSet	BooleanRadio controls.
Integer	LobIntegerInputSet	Input controls.
Money	LobGenericInputSet	Input controls.
Range	LobRangeInputSet	Range Input controls.
RangeRadio	LobRangeRadioInputSet	Range Radio Input controls.
Text	LobGenericInputSet	Input controls.
TextArea	LobTextAreaInputSet	TextArea Input controls.
TypeKey	Not supported.	
Year	LobIntegerInputSet	Input controls.
Date	LobDateInputSet	DateInput controls.
Decimal	LobDecimalInputSet	Input controls.

EffectiveDate

Availability data - the effective date for field availability based on the coverable.

ExpiryData

Availability Data - the expiration date for field availability based on the coverable.

Priority

Controls the display order.

Editable

PCF Attribute - editable attribute.

Available

PCF Attribute - available attribute.

Required

PCF Attribute - required attribute.

DefaultValue

PCF Attribute - the default value.

Max

PCF Attribute - the maximum value - integers and decimals WidgetTypes only.

Min

PCF Attribute - the minimum value - integers and decimals WidgetTypes only.

ForceRefresh

Triggers post on change (not required if the field is dependent - see dependency XML).

Version

Many attributes and aspects of a particular Column change over time as ISO changes the ERC. To manage this, there is a <Version> tag. Within a Version block, there can be several effective date segments defined by their start date. An example of a version block is shown below.

```
<Column name="ZipCodeOverride">
  <Version>
    <Jurisdiction>NY</Jurisdiction>
    <Label>entity.CA7ZoneRated.ZipCodeOverride</Label>
    <WidgetType>...</WidgetType>
    <EffectiveDate>2018-11-02</EffectiveDate>
    ...
  </Version>
  <Version>
    <Jurisdiction>NY</Jurisdiction>
    <Label>entity.CA7ZoneRated.ZipCodeOverride</Label>
    <Jurisdiction>NY</Jurisdiction>
    <Label>entity.CA7ZoneRated.ZipCodeOverride</Label>
    <WidgetType>...</WidgetType>
    <EffectiveDate>2019-07-02</EffectiveDate>
    ...
  </Version>
</Column>
```

In this example, there is a version of the schedule configuration for New York that is effective from November 2, 2018 until July 1, 2019. A second version for New York takes effect on July 2nd, 2019. In this example, assume that something within the “...” sections is the attribute that changed between versions.

Effective Date

The EffectiveDate tag can only be used within a Version block. Everything inside a specific Version block is the configuration that will be used from the date defined within the EffectiveDate start/end tags (specified in yyyy-mm-dd format) until the date defined in the next Version block. If there is no subsequent Version block, the end date is determined to be blank (the last Version block configuration will be used from the effective date forward).

Jurisdiction

The Jurisdiction tag, which is always found within a Version block, specifies that the configuration within the Version block only applies to that specific Jurisdiction. The following example shows a typical Version block with a Jurisdiction tag within it.

```
<Column name="ZipCodeOverride">
  <Version>
    <Jurisdiction>NY</Jurisdiction>
    <Label>entity.CA7ZoneRated.ZipCodeOverride</Label>
    <WidgetType>...</WidgetType>
    <EffectiveDate>2018-11-02</EffectiveDate>
    ...
  </Version>
```

In this case, the availability for the Zip Code Override field is assessed against this configuration only if the coverable is based in New York. The only valid content within the Jurisdiction tag is a standard two letter US State abbreviation.

